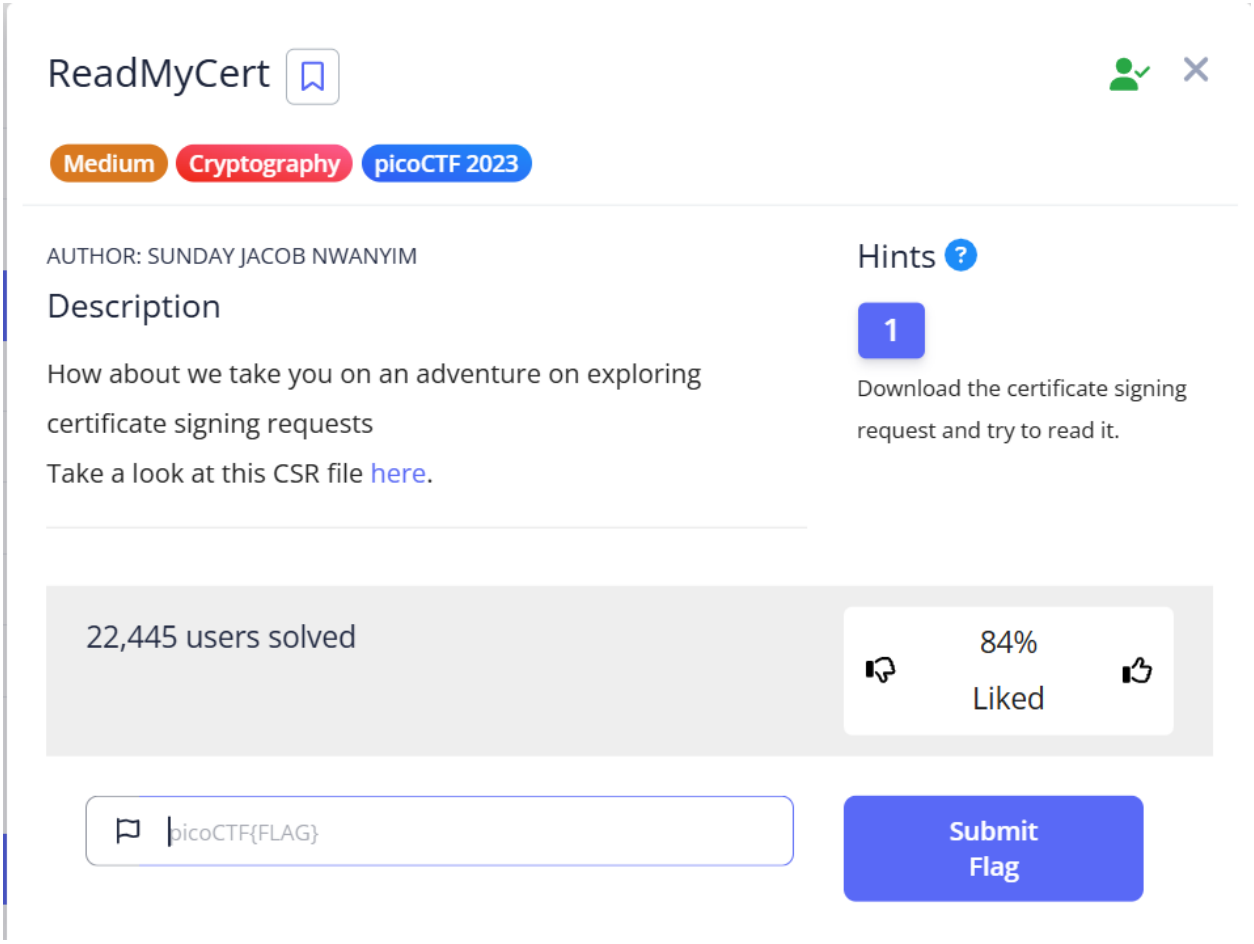
**HERNANDO, PANTINE B.** **BSIT-3**

**CHALLENGE – 20 – CRYPTOGRAPHY**




This challenge is a beginner-level forensics and cryptography task from **picoCTF 2024**. It introduces participants to **Certificate Signing Requests (CSRs)** and the standard tools used to inspect them.

### Description

The challenge provides a file named readmycert.csr. The objective is to extract the information contained within this request to find the flag.

**Solution Strategy**

A **CSR** is an encoded file used when applying for an SSL/TLS certificate. It contains identity information (Subject) and a public key. The information is typically encoded in **Base64** and wrapped in a **PEM** header.

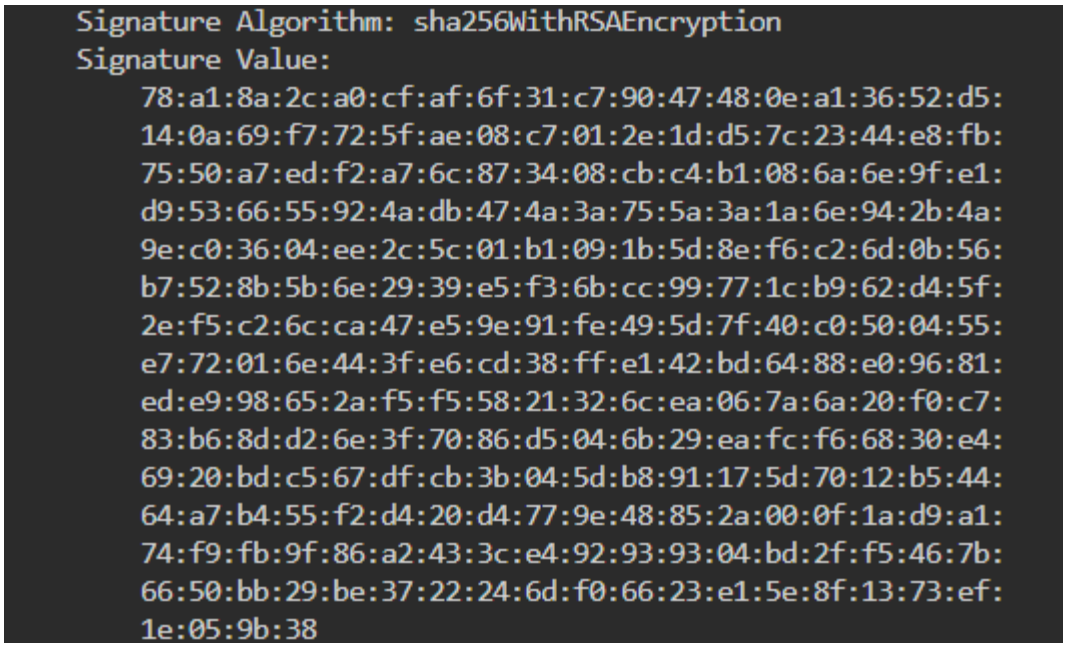
**Steps to solve:**

1. **Download the Artifact:** Retrieve the file using wget: wget https://artifacts.picoctf.net/c/423/readmycert.csr
2. **Inspect the File:** Running cat readmycert.csr shows the standard PEM format: -----BEGIN CERTIFICATE REQUEST----- followed by a block of Base64 data.

```
Pantine23-picoctf@webshell:~$ wget https://artifacts.picoctf.net/c/423/readmycert.csr
--2026-04-11 17:44:06--  https://artifacts.picoctf.net/c/423/readmycert.csr
Resolving artifacts.picoctf.net (artifacts.picoctf.net)... 3.170.131.72, 3.170.131.77, 3.170.131.33, ...
Connecting to artifacts.picoctf.net (artifacts.picoctf.net)|3.170.131.72|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 997 [application/octet-stream]
Saving to: 'readmycert.csr'

readmycert.csr                                      100%[=========================================

2026-04-11 17:44:06 (501 MB/s) - 'readmycert.csr' saved [997/997]

Pantine23-picoctf@webshell:~$ cat readmycert.csr
-----BEGIN CERTIFICATE REQUEST-----
MIICpzCCAY8CAQAwPDEmMCQGA1UEAwwdcGljb0NURntyZWFkX215Y2VydF81N2Y1
ODgzMn0xEjAQBgNVBCkMCWN0ZlBsYXllcjCCASIwDQYJKoZIhvcNAQEBBQADggEP
ADCCAQoCggEBANZde4bKP/88bBY0RM2b+EyGoxWqWsADa7QIPRZM4jTAxPTC39Ld
6iDZwfA6Cu33KvkZbu1JpAFk/6O/lY+iwSCcZnBTp1p+Skn/BpIwW7KBEjnczulA
c/u4GYQgpU5Pxxd/gvOHLNtWHw8FjcHAV78Y23cwwfO1Gfae5eYrxHMa/nCiQmjC
9GwRsj2+cPmWiyFs1ntLREBGUKWBIHGoTR+ZMXv9aBeasIUlzWap/4ZsSOqoqzAL
3geZ9TfWd/pHtYgqA1jV60ogmWD2LKMU9F4s+5dJveO/5kV7kkpk+7VX3xlE1t/S
0/ThtcNU51WdfmFREr2hCUJgicQHbkkwq00CAwEAAaAmMCQGCSqGSIb3DQEJDjEX
MBUwEwYDVR0lBAwwCgYIKwYBBQUHAwIwDQYJKoZIhvcNAQELBQADggEBAHihiiyg
z69vMceQR0gOoTZS1RQKafdyX64IxwEuHdV8I0To+3VQp+3yp2yHNAjLxLEIam6f
4dlTZlWSSttHSjp1WjoabpQrSp7ANgTuLFwBsQkbXY72wm0LVrdSi1tuKTnl82vM
mXccuWLUXy71wmzKR+Wekf5JXX9AwFAEVedyAW5EP+bNOP/hQr1kiOCWge3pmGUq
9fVYITJs6gZ6aiDwx4O2jdJuP3CG1QRrKer89mgw5GkgvcVn38s7BF24kRddcBK1
RGSntFXy1CDUd55IhSoADxrZoXT5+5+GokM85JKTkwS9L/VGe2ZQuym+NyIkbfBm
I+FejxNz7x4Fmzg=
-----END CERTIFICATE REQUEST-----
```

3. **Decode the Request:** To read the metadata inside the CSR, use the **OpenSSL** library. The following command decodes the file into a human-readable format: openssl req -in readmycert.csr -noout -text
    - -in: Specifies the input file.
    - -noout: Prevents the command from outputting the encoded version again.
    - -text: Displays the text content of the request.

```
-----END CERTIFICATE REQUEST-----
Pantine23-picoctf@webshell:~$ openssl req -in readmycert.csr -noout -text
Certificate Request:
    Data:
        Version: 1 (0x0)
        Subject: CN = picoCTF{read_mycert_57f58832}, name = ctfPlayer
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:d6:5d:7b:86:ca:3f:ff:3c:6c:16:34:44:cd:9b:
                    f8:4c:86:a3:15:aa:5a:c0:03:6b:b4:08:3d:16:4c:
                    e2:34:c0:c4:f4:c2:df:d2:dd:ea:20:d9:c1:f0:3a:
                    0a:ed:f7:2a:f9:19:6e:ed:49:a4:01:64:ff:a3:bf:
                    95:8f:a2:c1:20:9c:66:70:53:a7:5a:7e:4a:49:ff:
                    06:92:30:5b:b2:81:12:39:dc:ce:e9:40:73:fb:b8:
                    19:84:20:a5:4e:4f:c7:17:7f:82:f3:87:2c:db:56:
                    1f:0f:05:8d:c1:c0:57:bf:18:db:77:30:c1:f3:b5:
                    19:f6:9e:e5:e6:2b:c4:73:1a:fe:70:a2:42:68:c2:
                    f4:6c:11:b2:3d:be:70:f9:96:8b:21:6c:d6:7b:4b:
                    44:40:46:50:a5:81:20:71:a8:4d:1f:99:31:7b:fd:
                    68:17:9a:b0:85:25:cd:66:a9:ff:86:6c:48:ea:a8:
                    ab:30:0b:de:07:99:f5:37:d6:77:fa:47:b5:88:2a:
                    03:58:d5:eb:4a:20:99:60:f6:2c:a3:14:f4:5e:2c:
                    fb:97:49:bd:e3:bf:e6:45:7b:92:4a:64:fb:b5:57:
                    df:19:44:d6:df:d2:d3:f4:e1:b5:c3:54:e7:55:9d:
                    7e:61:51:12:bd:a1:09:42:60:89:c4:07:6e:49:30:
                    ab:4d
                Exponent: 65537 (0x10001)
        Attributes:
            Requested Extensions:
                X509v3 Extended Key Usage:
                    TLS Web Client Authentication
    Signature Algorithm: sha256WithRSAEncryption
```




**Analysis of Findings**

When the command is executed, OpenSSL displays the Certificate Request data. By looking at the **Subject** field, we can see the identity information associated with the request:

**Subject: CN = picoCTF{read_mycert_57f58832}, name = ctfPlayer**

The **Common Name (CN)** field is a standard place for organizers to hide flags in certificate-related challenges.

---

**Final Result**

The flag was found embedded in the CN (Common Name) attribute of the Subject.

**Flag:**

picoCTF{read_mycert_57f58832}

**Short Reflection**

The "readmycert" challenge highlights a critical lesson in digital forensics: **information is often hidden in plain sight within standard protocols.**

- **Metadata Awareness:** In security, the "container" is often as important as the content. Understanding how to use industry-standard tools like **OpenSSL** allows an investigator to peer inside administrative files (like CSRs or Certificates) to find leaked information or intentionally placed flags.

- **The Power of the Subject Field:** The **Subject** field is a structured repository of identity. While it is intended for names and organizations, it can be manipulated to carry any string, making it a common target for inspection in CTF challenges.
- **Protocol Literacy:** This challenge reinforces that being a successful security professional requires "protocol literacy"—the ability to recognize standard file headers (like PEM) and knowing which tools to apply to decode them.